

数据结构实验报告

软件学院 软件工程二班 李宗杭 2311451

实验题目：KMP&BM 性能比较

一、实验说明：

本实验旨在通过构造不同场景下的主串和模式字符串，对比 KMP 算法（Knuth-Morris-Pratt 算法）和 BM 算法（Boyer-Moore 算法）在字符串匹配过程中的性能表现，包括字符比较次数和搜索时间等指标。

KMP 算法是一种改进的字符串匹配算法，由 D.E.Knuth、J.H.Morris 和 V.R.Pratt 同时发现，目的是通过预处理模式串，构建一个部分匹配表，然后利用该表进行模式匹配，从而减少不必要的比较次数，提高匹配效率。

KMP 算法的时间复杂度为 $O(m+n)$ ，其中 m 和 n 分别表示模式串和主串的长度，预处理模式串构建 next 数组需要 $O(m)$ 的时间，匹配过程需要 $O(n)$ 的时间。

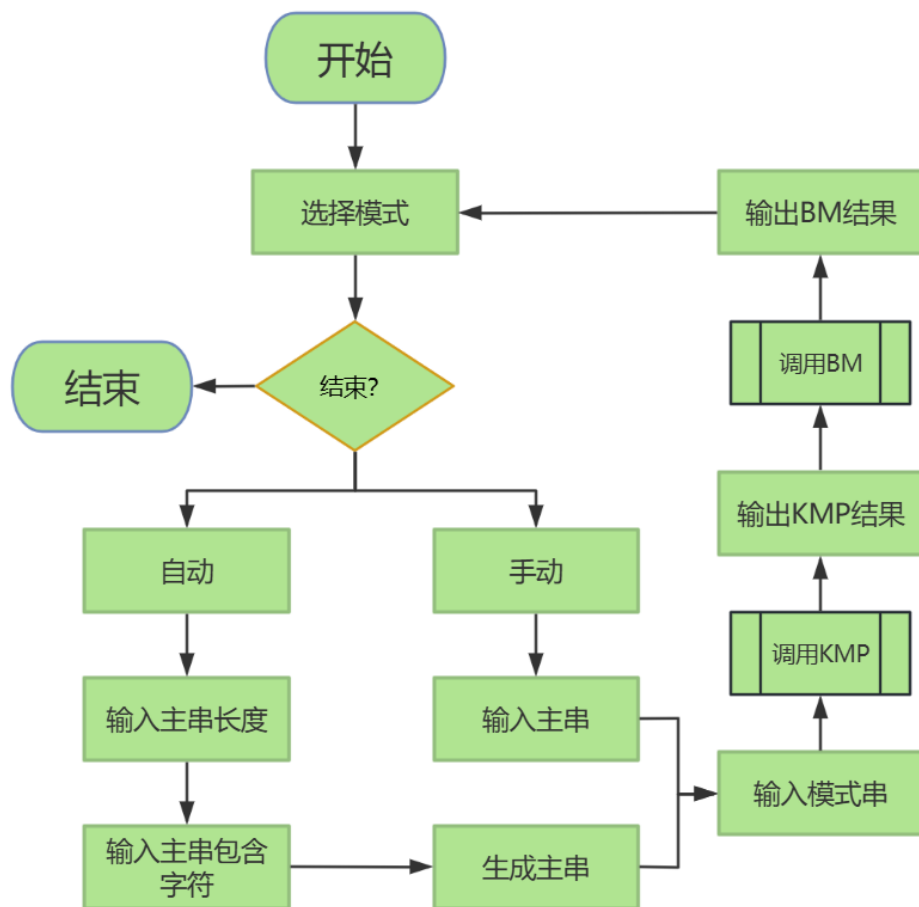
BM 算法由 Bob Boyer 和 J Strother Moore 设计并提出。它与其他字符串匹配算法的主要区别在于其匹配顺序和启发式策略。BM 算法的核心思想是从右向左进行模式匹配，并利用坏字符规则和好后缀规则来计算模式串的移动偏移量。这两种规则分别基于不匹配字符的位置和已匹配后缀的信息来优化匹配过程。

BM 算法的时间复杂度在最坏情况下为 $O(mn)$ ，其中 m 和 n 分别表示模式串和文本串的长度，其平均时间复杂度通常远低于 $O(mn)$ ，在最好情况下可以达到 $O(n/m)$ 。

相对而言，KMP 算法的优势体现在模式串较短且需要频繁匹配时，BM 算法在处理大规模数据和长模式串时具有显著优势，这也是本次实验希望达到的预期效果。

本次实验在同一程序中分别实现 kmp 算法和 bm 算法。每轮测试首先从键盘接收一个字符来选择手动输入主串或随机生成主串，若选择随机生成则还需依次输入主串长度以及包含的字符类型。随后接收一个模式串，然后依次输出使用 kmp 算法以及 bm 算法进行字符串搜索过程中的字符比较次数以及搜索时间。

二、流程图：



三、测试结果

1、01 串

主串大小	模串大小	实验结果	KMP		BM		快
			比较次数	运行时间/ms	比较次数	运行时间/ms	
短 100	短 1	(靠前)Place: 6	11	0.0204	7	0.0224	KMP
		(靠后)Place: 92	183	0.0241	93	0.0265	KMP
		No Find	200	0.0238	100	0.0321	KMP
	中 10	(靠前)Place: 11	30	0.0218	23	0.0273	KMP
		(靠后)Place: 80	156	0.0269	79	0.0289	KMP
		No Find	174	0.0288	51	0.0308	KMP
	长 50	(靠前)Place: 4	56	0.0425	55	0.0561	KMP
		(靠后)Place: 50	124	0.0283	65	0.1526	KMP
		No Find	151	0.0308	14	0.0334	KMP
长 100,000	短 10	(靠前)Place: 144	296	0.1066	91	0.0295	BM
		(靠后)Place: 99527	199062	2.0738	19921	0.5339	BM
		No Find	175078	6.9598	65695	4.2247	BM
	中 500	(靠前)Place: 1576	3251	0.2702	629	0.1922	BM
		(靠后)Place: 97221	152219	11.884	11133	1.4524	BM
		No Find	156029	9.8502	10550	1.3037	BM
	长 10,000	(靠前)Place: 260	10443	1.2106	10057	2.6984	KMP
		(靠后)Place: 88698	156434	11.798	16596	4.3627	BM
		No Find	165301	11.621	7099	2.1447	BM

2、包含 10 种字符

主串大小	模串大小	实验结果	KMP		BM		快
			比较次数	运行时间/ms	比较次数	运行时间/ms	
短 100	短 1	(靠前)Place: 6	11	0.0186	7	0.0246	KMP
		(靠后)Place: 63	126	0.0253	38	0.0230	BM
		No Find	200	0.0238	100	0.0285	KMP
	中 10	(靠前)Place: 3	14	0.0213	13	0.0243	KMP
		(靠后)Place: 81	168	0.0253	27	0.0311	KMP
		No Find	200	0.0242	21	0.0195	BM
	长 50	(靠前)Place: 5	58	0.0271	52	0.0351	KMP
		(靠后)Place: 46	140	0.0280	56	0.0299	KMP
		No Find	199	0.0255	4	0.0300	KMP
长 100,000	短 10	(靠前)Place: 12	32	0.1108	13	0.0313	BM
		(靠后)Place: 99987	198902	4.5723	13636	1.3181	BM
		No Find	198922	4.5790	20366	0.9335	BM
	中 500	(靠前)Place: 128	754	0.0976	518	0.1094	KMP
		(靠后)Place: 98057	195545	4.8154	11248	0.7634	BM
		No Find	198904	4.9498	10982	1.0762	BM
	长 10,000	(靠前)Place: 250	10496	1.0752	10031	1.7459	KMP
		(靠后)Place: 78159	166147	5.1619	14375	1.7281	BM
		No Find	199779	5.6364	6261	1.8415	BM

3、包含 ASCII 32~126(共 95 个)字符

主串大小	模串大小	实验结果	KMP		BM		快
			比较次数	运行时间/ms	比较次数	运行时间/ms	
短 100	短 1	(靠前)Place: 7	13	0.0223	8	0.0292	KMP
		(靠后)Place: 69	137	0.0234	70	0.0250	KMP
		No Find	200	0.0229	100	0.0311	KMP
	中 10	(靠前)Place: 8	24	0.0185	13	0.0192	KMP
		(靠后)Place: 77	162	0.0224	19	0.0279	KMP
		No Find	200	0.0242	11	0.0301	KMP
	长 50	(靠前)Place: 5	58	0.0170	52	0.0275	KMP
		(靠后)Place: 46	140	0.0238	52	0.0362	KMP
		No Find	200	0.0257	2	0.0282	KMP
长 100,000	短 10	(靠前)Place: 24	56	0.0944	14	0.0459	BM
		(靠后)Place: 99918	199835	4.5776	10639	1.1256	BM
		No Find	199990	4.4555	10655	1.0451	BM
	中 500	(靠前)Place: 247	992	0.0550	506	0.0473	BM
		(靠后)Place: 98071	196631	4.6553	1608	0.2284	BM
		No Find	199990	4.8086	1137	0.2073	BM
	长 10,000	(靠前)Place: 251	10499	1.1800	10004	1.6893	KMP
		(靠后)Place: 89873	189727	5.0400	10988	0.9969	BM
		No Find	199988	5.2610	901	1.3230	BM

四、分析与探讨

1、测试结果分析

综合观察三个表格来看，无论字符集为何种，呈现的总体趋势基本一致。

当主串较短时，无论模式串相对于主串的长短，KMP 算法与 BM 算法的运行时间相差不大，但 KMP 总是略优于 BM；当主串较长时，无论模式串相对于主串的长短，BM 算法的运行时间总是优于 KMP 算法。无论主串与模式串长度如何，从本次测试结果可以看出 BM 算法的比较次数总是少于 KMP 算法。

主串的绝对长度越长，模式串的相对长度越长，以及无法找到模式串时，BM 算法与 KMP 算法的运行时间与比较次数的差距就越大，BM 算法优势越显著。

也可看出，虽然在长主串时 BM 算法更具优势，但当模式串在主串中的位置较为靠前时，KMP 算法的性能又赶超 BM 算法。

2、探讨存在的问题与改进措施

此次测试数据量较小，每种情况仅测试了一次，若要得到更为普遍的结论，应该将每种情况都多次测试，刨除偶然情况，得到平均情况。

应该在此长主串之上再加一类更长的主串进行测试以便得出更普遍的规律，由于电脑性能原因放弃了此项测试。

总的来说本次测试由于测试量较小，仅能初步得出主串长短以及模式串在主串中的位置对于算法性能可能产生的影响，尚难以得出模式串相对于主串的大小以及字符串的字符集所产生的影响。

不过本次实验也可得出一些结论：当选择算法时，若主串较小，可以选择 KMP，主串较长则可选择 BM；当模式串在主串中位置靠前时可以考虑 KMP 算法。

最后，通过本次实验，我深入理解了 KMP 和 BM 的实现原理，掌握了相应的算法和数据结构，并且通过编写程序实现了该功能。此外我还额外学到了一个生成随机字符串的方法以及计算程序段运行时间的方法。

在实验过程中，我提高了编程能力和解决问题的能力，加深了对算法和数据结构的理解。未来将继续学习和探索更多有关表达式转换和算法设计的知识，不断提升自己的能力。

```

#include<iostream>
#include<string>
#include<chrono>
#include<vector>
#include<ctime>
#include<random>
using namespace std;

unsigned long long kmptimes, bmtimes;//比较次数

//KMP 预处理next[]
void getnext(string t, int next[])
{
    int i = 0, j = -1;
    next[0] = -1;
    while (i < t.length() - 1)
    {
        if (j == -1 || t[i] == t[j])
        {
            ++i;
            ++j;
            next[i] = j;
        }
        else
        {
            j = next[j];
        }
    }
}

//KMP 实现
int kmp(string S, string T)
{
    int i = 0, j = 0;
    int* next = new int[T.length()];
    getnext(T, next);
    int S1 = S.length(), T1 = T.length();
    while (i < S1 && j < T1)
    {
        kmptimes++;
        if (j == -1 || S[i] == T[j])
        {
            ++i;
            ++j;
        }
        else
        {
            j = next[j];
        }
    }
}

```

```

        if (j >= Tl)
            return i - Tl;
        else
            return -1;
    }

```

//BM 预处理坏字符

```

void GetBC(vector<int>& BC, const string pstr)
{
    int len = pstr.size();
    // BC 传过来的时候已经全是-1 了
    for (int i = 0; i < len; i++)
    {
        BC[pstr[i]] = i;
    }
}

```

//BM 预处理后缀

```

void getsuffix(vector<int>& suffix, const string& pstr)
{
    int len = pstr.size();
    suffix[len - 1] = len;
    for (int i = len - 2; i >= 0; i--)
    {
        for (int j = 0; j <= i && pstr[i - j] == pstr[len - j - 1]; j++)
        {
            suffix[i] = j + 1;
        }
    }
}

```

//BM 预处理好后缀

```

void getGS(vector<int>& GS, const vector<int>& suffix, const string& pstr)
{
    int len = pstr.size();
    // GS 构建时已经被初始化为n 个Len
    // 先找前缀
    for (int i = len - 1; i >= 0; i--)
    {
        if (suffix[i] == i + 1)
        {
            for (int j = 0; j < len - 1 - i; j++)
                if (GS[j] == len)
                    GS[j] = len - i - 1;
        }
    }
    // 找串中子串
    for (int i = 0; i < len - 1; i++)
    {
        GS[len - 1 - suffix[i]] = len - 1 - i;
    }
}

```

```

}
//BM 实现
int BM_Match(const string& tstr, const string& pstr)
{
    int len = pstr.size();
    vector<int> BC(256, -1);
    vector<int> suffix(len, 0);
    vector<int> GS(len, len);
    getsuffix(suffix, pstr);
    getGS(GS, suffix, pstr);
    GetBC(BC, pstr);
    int n = tstr.size();
    int i = len - 1, j = len - 1;
    int tlen = tstr.size();
    int shift = 0;
    int pos = len - 1;
    while (shift + len - 1 < tlen)
    {
        bmtimes++;
        for (pos = len - 1; pos >= 0 && pstr[pos] == tstr[shift + pos]; pos--,
bmtimes++);
        if (pos == -1)
            break;
        else
            shift += max(pos - BC[tstr[shift + pos]], GS[pos]);
    }
    return shift + len - 1 < tlen ? shift : -1;
}

int main()
{
    for(int i=1;;i++)
    {
        kmptimes = bmtimes = 0; //比较次数重置
        int mode; //输入模式
        cout << "第" << i << "次测试: " << endl;
        cout << "手动: 1   自动: 2   结束: 0" << endl;
        cin >> mode;
        //mode = 1;
        //结束
        if (mode == 0)
        {
            return 0;
        }

        string ss, st; //主串, 模式串

        //手动
        if (mode == 1)

```

```

{
    cin.ignore();
    cout << "主串: ";
    getline(cin, ss);
}

//自动
if (mode == 2)
{
    size_t ssn; //主串长度
    string charset; //包含字符
    cout << "主串长度: ";
    cin >> ssn;
    //ssn = 100000;
    cout << ssn << endl;
    cin.ignore();
    cout << "包含字符: ";
    getline(cin, charset);
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, charset.size() - 1);
    ss.reserve(ssn);
    for (size_t i = 0; i < ssn; i++)
    {
        ss += charset[dis(gen)];
    }
    cout << "主串: " << ss << endl;
}

cout << "模串: ";
cin.ignore();
getline(cin, st);
int pkmp; //kmp 位置
int pbm; //bm 位置

//kmp 结果
cout << "KMP:" << endl;
auto kmpstart = chrono::high_resolution_clock::now(); //计时开始
pkmp = kmp(ss, st);
auto kmpend = chrono::high_resolution_clock::now(); //计时结束
chrono::duration<double, milli> kmp_elapsed_milliseconds = kmpend -
kmpstart; //运行时间
cout << "查找结果: ";
if (pkmp == -1)
    cout << "No find" << endl;
else
    cout << "Place:" << pkmp + 1 << endl;
cout << "比较次数: " << kmptimes << endl;
cout << "运行时间: " << kmp_elapsed_milliseconds.count() << "ms" << endl;

```

```

    cout << "BM:" << endl;
    auto bmstart = chrono::high_resolution_clock::now();
    pbm = BM_Match(ss, st);
    auto bmend = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> bm_elapsed_milliseconds = bmend - bmstart;
    cout << "查找结果: ";
    if (pbm == -1)
        cout << "No find" << endl;
    else
        cout << "Place:" << pbm+1 << endl;
    cout << "比较次数: " << bmtimes << endl;
    cout << "运行时间: " << bm_elapsed_milliseconds.count() << "ms" << endl;
    cout << endl;
}

return 0;
}

```

```

#include<iostream>
#include<string>
#include<chrono>
#include<vector>
#include<time>
#include<random>
using namespace std;

unsigned long long haptimes, haptimes0; //播放次数

//判断是否是回文
void getans(string s, int start[])
{
    int i = 0, j = s.length() - 1;
    while (i < j)
    {
        if (s[i] == s[j])
        {
            ++i;
            --j;
        }
        else
        {
            j = next[i];
        }
    }
}

//判断长度
void hup(string S, string T)
{
    int i = 0, j = 0;
    int next = new int[T.length()];
    getans(T, next);
    int S1 = S.length(), T1 = T.length();
    while (i < S1 && j < T1)
    {
        haptimes++;
        if (j == -1 || S[i] == T[j])
        {
            ++i;
            ++j;
        }
        else
        {
            j = next[j];
        }
    }
    if (j >= T1)
        return i - T1;
    else
        return -1;
}

//判断处理前缀字符串
void GetHC(vector<cin>& HC, const string ptr)
{
    int len = ptr.size();
    // 前缀字符串的初始化为T个空字符串
    for (int i = 0; i < len; i++)
        HC.push(i, 1);
}

//判断处理后缀字符串
void getSuffix(vector<cin>& suffix, const string ptr)
{
    int len = ptr.size();
    suffix[len - 1] = len;
    for (int i = len - 2; i >= 0; i--)
    {
        for (int j = 0; j < i && ptr[i - j] == ptr[len - j - 1]; j++)
            suffix[i] = j + 1;
    }
}

//判断处理中间字符串
void getG(vector<cin>& GS, const vector<cin>& suffix, const string ptr)
{
    int len = ptr.size();
    // 中间字符串的初始化为T个空字符串
    // 东边和南边
    for (int i = len - 1; i >= 0; i--)
    {
        if (suffix[i] == i + 1)
        {
            for (int j = 0; j < len - i - 1; j++)
            {
                GS[i] = len - i - 1;
            }
        }
    }
    // 西边和北边
    for (int i = 0; i < len - 1; i++)
    {
        GS[len - 1 - suffix[i]] = len - 1 - i;
    }
}

//判断结果
void BM_Match(const string tstr, const string ptr)
{
    int len = ptr.size();
    vector<cin> GS(len, -1);
    vector<cin> suffix(len, 1);
    vector<cin> GS(len, len);
    getans(suffix, ptr);
    getGS(GS, ptr);
    int n = ptr.size();
    int i = len - 1, j = len - 1;
    int times = ptr.size();
    int shift = 0;
    int pos = len - 1;
    while (shift < len - 1 < times)
    {
        haptimes++;
        for (pos = len - 1; pos >= 0 && ptr[pos] == tstr[shift + pos]; pos--, haptimes++)
            if (pos == -1)
                break;
        shift = max(pos - HC[tstr[shift + pos]], GS[pos]);
    }
    return shift < len - 1 < times ? shift : -1;
}

int main()
{
    for (int i = 1; i <= 4; i++)
    {
        haptimes0 = haptimes = 0; //播放次数重置
        cout << "输入：";
        string s;
        while (s != "q")
        {
            cin >> s;
            if (s == "q")
                break;
            if (s == "a")
                continue;
            if (s == "b")
                continue;
            if (s == "c")
                continue;
            if (s == "d")
                continue;
            if (s == "e")
                continue;
            if (s == "f")
                continue;
            if (s == "g")
                continue;
            if (s == "h")
                continue;
            if (s == "i")
                continue;
            if (s == "j")
                continue;
            if (s == "k")
                continue;
            if (s == "l")
                continue;
            if (s == "m")
                continue;
            if (s == "n")
                continue;
            if (s == "o")
                continue;
            if (s == "p")
                continue;
            if (s == "q")
                continue;
            if (s == "r")
                continue;
            if (s == "s")
                continue;
            if (s == "t")
                continue;
            if (s == "u")
                continue;
            if (s == "v")
                continue;
            if (s == "w")
                continue;
            if (s == "x")
                continue;
            if (s == "y")
                continue;
            if (s == "z")
                continue;
            if (s == "0")
                continue;
            if (s == "1")
                continue;
            if (s == "2")
                continue;
            if (s == "3")
                continue;
            if (s == "4")
                continue;
            if (s == "5")
                continue;
            if (s == "6")
                continue;
            if (s == "7")
                continue;
            if (s == "8")
                continue;
            if (s == "9")
                continue;
            if (s == "A")
                continue;
            if (s == "B")
                continue;
            if (s == "C")
                continue;
            if (s == "D")
                continue;
            if (s == "E")
                continue;
            if (s == "F")
                continue;
            if (s == "G")
                continue;
            if (s == "H")
                continue;
            if (s == "I")
                continue;
            if (s == "J")
                continue;
            if (s == "K")
                continue;
            if (s == "L")
                continue;
            if (s == "M")
                continue;
            if (s == "N")
                continue;
            if (s == "O")
                continue;
            if (s == "P")
                continue;
            if (s == "Q")
                continue;
            if (s == "R")
                continue;
            if (s == "S")
                continue;
            if (s == "T")
                continue;
            if (s == "U")
                continue;
            if (s == "V")
                continue;
            if (s == "W")
                continue;
            if (s == "X")
                continue;
            if (s == "Y")
                continue;
            if (s == "Z")
                continue;
            if (s == "0")
                continue;
            if (s == "1")
                continue;
            if (s == "2")
                continue;
            if (s == "3")
                continue;
            if (s == "4")
                continue;
            if (s == "5")
                continue;
            if (s == "6")
                continue;
            if (s == "7")
                continue;
            if (s == "8")
                continue;
            if (s == "9")
                continue;
            if (s == "A")
                continue;
            if (s == "B")
                continue;
            if (s == "C")
                continue;
            if (s == "D")
                continue;
            if (s == "E")
                continue;
            if (s == "F")
                continue;
            if (s == "G")
                continue;
            if (s == "H")
                continue;
            if (s == "I")
                continue;
            if (s == "J")
                continue;
            if (s == "K")
                continue;
            if (s == "L")
                continue;
            if (s == "M")
                continue;
            if (s == "N")
                continue;
            if (s == "O")
                continue;
            if (s == "P")
                continue;
            if (s == "Q")
                continue;
            if (s == "R")
                continue;
            if (s == "S")
                continue;
            if (s == "T")
                continue;
            if (s == "U")
                continue;
            if (s == "V")
                continue;
            if (s == "W")
                continue;
            if (s == "X")
                continue;
            if (s == "Y")
                continue;
            if (s == "Z")
                continue;
            if (s == "0")
                continue;
            if (s == "1")
                continue;
            if (s == "2")
                continue;
            if (s == "3")
                continue;
            if (s == "4")
                continue;
            if (s == "5")
                continue;
            if (s == "6")
                continue;
            if (s == "7")
                continue;
            if (s == "8")
                continue;
            if (s == "9")
                continue;
            if (s == "A")
                continue;
            if (s == "B")
                continue;
            if (s == "C")
                continue;
            if (s == "D")
                continue;
            if (s == "E")
                continue;
            if (s == "F")
                continue;
            if (s == "G")
                continue;
            if (s == "H")
                continue;
            if (s == "I")
                continue;
            if (s == "J")
                continue;
            if (s == "K")
                continue;
            if (s == "L")
                continue;
            if (s == "M")
                continue;
            if (s == "N")
                continue;
            if (s == "O")
                continue;
            if (s == "P")
                continue;
            if (s == "Q")
                continue;
            if (s == "R")
                continue;
            if (s == "S")
                continue;
            if (s == "T")
                continue;
            if (s == "U")
                continue;
            if (s == "V")
                continue;
            if (s == "W")
                continue;
            if (s == "X")
                continue;
            if (s == "Y")
                continue;
            if (s == "Z")
                continue;
            if (s == "0")
                continue;
            if (s == "1")
                continue;
            if (s == "2")
                continue;
            if (s == "3")
                continue;
            if (s == "4")
                continue;
            if (s == "5")
                continue;
            if (s == "6")
                continue;
            if (s == "7")
                continue;
            if (s == "8")
                continue;
            if (s == "9")
                continue;
            if (s == "A")
                continue;
            if (s == "B")
                continue;
            if (s == "C")
                continue;
            if (s == "D")
                continue;
            if (s == "E")
                continue;
            if (s == "F")
                continue;
            if (s == "G")
                continue;
            if (s == "H")
                continue;
            if (s == "I")
                continue;
            if (s == "J")
                continue;
            if (s == "K")
                continue;
            if (s == "L")
                continue;
            if (s == "M")
                continue;
            if (s == "N")
                continue;
            if (s == "O")
                continue;
            if (s == "P")
                continue;
            if (s == "Q")
                continue;
            if (s == "R")
                continue;
            if (s == "S")
                continue;
            if (s == "T")
                continue;
            if (s == "U")
                continue;
            if (s == "V")
                continue;
            if (s == "W")
                continue;
            if (s == "X")
                continue;
            if (s == "Y")
                continue;
            if (s == "Z")
                continue;
            if (s == "0")
                continue;
            if (s == "1")
                continue;
            if (s == "2")
                continue;
            if (s == "3")
                continue;
            if (s == "4")
                continue;
            if (s == "5")
                continue;
            if (s == "6")
                continue;
            if (s == "7")
                continue;
            if (s == "8")
                continue;
            if (s == "9")
                continue;
            if (s == "A")
                continue;
            if (s == "B")
                continue;
            if (s == "C")
                continue;
            if (s == "D")
                continue;
            if (s == "E")
                continue;
            if (s == "F")
                continue;
            if (s == "G")
                continue;
            if (s == "H")
                continue;
            if (s == "I")
                continue;
            if (s == "J")
                continue;
            if (s == "K")
                continue;
            if (s == "L")
                continue;
            if (s == "M")
                continue;
            if (s == "N")
                continue;
            if (s == "O")
                continue;
            if (s == "P")
                continue;
            if (s == "Q")
                continue;
            if (s == "R")
                continue;
            if (s == "S")
                continue;
            if (s == "T")
                continue;
            if (s == "U")
                continue;
            if (s == "V")
                continue;
            if (s == "W")
                continue;
            if (s == "X")
                continue;
            if (s == "Y")
                continue;
            if (s == "Z")
                continue;
            if (s == "0")
                continue;
            if (s == "1")
                continue;
            if (s == "2")
                continue;
            if (s == "3")
                continue;
            if (s == "4")
                continue;
            if (s == "5")
                continue;
            if (s == "6")
                continue;
            if (s == "7")
                continue;
            if (s == "8")
                continue;
            if (s == "9")
                continue;
            if (s == "A")
                continue;
            if (s == "B")
                continue;
            if (s == "C")
                continue;
            if (s == "D")
                continue;
            if (s == "E")
                continue;
            if (s == "F")
                continue;
            if (s == "G")
                continue;
            if (s == "H")
                continue;
            if (s == "I")
                continue;
            if (s == "J")
                continue;
            if (s == "K")
                continue;
            if (s == "L")
                continue;
            if (s == "M")
                continue;
            if (s == "N")
               
```